INDIAN INSTITUTE OF INFORMATION TECHNOLOGY
TIRUCHIRAPPALLI
Department of Computer Science and Engineering

Automated Plagiarism Checker

Detecting similarity between text and source code files using N-gram analysis

Mini Project Report
Object-Oriented Programming with C++

Submitted by

Charan Teja Konapala (Roll No. 251114)
Rushitha Gangavarapu (Roll No. 251121)

Under the guidance of
Dr. Anoop Jacob Thomas

Academic Year 2026

Contents

1 Automated Plagiarism Checker **1**

1.1 Introduction and Problem Statement . . . 1
1.1.1 Objectives . . . 1
1.2 Scope and Feature Set . . . 2
1.2.1 In scope . . . 2
1.2.2 Out of scope . . . 2
1.2.3 Assumptions and constraints . . . 2
1.3 Background: N-Gram Similarity . . . 3
1.3.1 N-grams and pre-processing . . . 3
1.3.2 Similarity measure . . . 3
1.3.3 Similarity categories . . . 3
1.4 System Architecture and Design . . . 3
1.4.1 Separation of concerns . . . 3
1.4.2 Modules . . . 4
1.4.3 Processing flow . . . 4
1.5 Object-Oriented Architecture . . . 6
1.5.1 Class design and associations . . . 6
1.5.2 Access modifiers . . . 6
1.5.3 Composition, aggregation and inheritance . . . 7
1.5.4 UML class diagram . . . 7
1.6 Advanced C++ Mechanics . . . 8
1.6.1 Templates . . . 8
1.6.2 Exception handling and error checking . . . 9
1.6.3 Operator overloading . . . 10
1.6.4 Memory management . . . 10
1.7 Frontend and Backend Integration . . . 10
1.7.1 Frontend . . . 10
1.7.2 Backend . . . 12
1.7.3 Persistent storage and file I/O . . . 12
1.8 Implementation Summary . . . 16
1.9 Version Control (Git) . . . 17
1.10 Testing and Results . . . 17
1.10.1 Test data . . . 17
1.10.2 Observed results . . . 18
1.10.3 Evaluation . . . 18
1.10.4 Error and edge-case testing . . . 19
1.11 Limitations . . . 19
1.12 Future Enhancements . . . 19

1.13 Conclusion 20
1.14 References 20

List of Figures

1.1 Processing flow of the back end, from the input files to the saved reports. 5
1.2 Class relationship diagram of the back end. 8
1.3 Main window after comparing 13 files (78 pairs). 11
1.4 Side-by-side fragment viewer for student12.cpp and student3.cpp (39.59%, Moderate). 12
1.5 How a result is stored, categorised and written to the TXT and JSON reports. 13
1.6 Export Results to CSV save dialog. 14
1.7 plagiarism_report.txt, with one block per file pair (files, similarity, category and matching fragments). 15
1.8 plagiarism_report.json, showing the project name, n_gram_size of 3 and the results array. 15
1.9 plagiarism_results.csv, one row per pair (student11 and student8 at 100.00, Very High; student12 and student3 at 39.59, Moderate). 16
1.10 The public GitHub repository on branch main, with 51 commits. 17

List of Tables

1.1 Similarity categories used by the report. 3
1.2 Layers of the system and the classes in each layer. 4
1.3 Classes, data and public interface of the back end. 6
1.4 Persistent report formats. 13
1.5 Implementation summary. 16
1.6 Observed results on the 13-file test set. 18
1.7 Error and edge-case testing. 19

Chapter 1

Automated Plagiarism Checker

Charan Teja Konapala (251114) *Rushitha Gangavarapu (251121)*

Abstract

Automated Plagiarism Checker is a C++ desktop application that compares multiple text and source code files and measures how similar they are. It uses word-level N-gram analysis (N = 3 by default, selectable as 2, 3 or 5 words) to turn every file into a set of overlapping word sequences, compares those sets, and reports a similarity percentage for every pair of files. Each result is classified into a category (Low, Moderate, High or Very High), and the matching text fragments are shown so that a reviewer can see exactly why two files were flagged. The system has a Qt-based graphical front end, a C++ back end built from separate, independent classes, and persistent storage through exportable TXT, JSON and CSV reports.

1.1 Introduction and Problem Statement

In academic settings, programming assignments and written work are routinely submitted by dozens of students. Manually checking every submission against every other submission is slow, inconsistent and practically impossible as the number of files grows: for n files there are $n(n-1)/2$ pairs to compare. Students can also hide copying by renaming variables, changing spacing or reordering lines, which makes simple visual inspection unreliable.

The problem addressed by this project is therefore: to design and implement an automated, offline desktop tool that accepts a collection of text or source code files, quantifies the similarity between every pair of files, and presents the results in a clear, reviewable and exportable form.

1.1.1 Objectives

- Load several text and source code files from a folder selected by the user.
- Normalise the content so that formatting differences do not hide similarity.
- Generate N-grams for each file and compute a similarity percentage for each pair.

- Categorise the similarity level and list the matching text fragments.
- Present results in a GUI and export them to persistent report files (TXT, JSON, CSV).
- Apply core object-oriented principles and robust error handling in a maintainable, modular code base.

1.2 Scope and Feature Set

1.2.1 In scope

- **Pairwise comparison:** text and source code files compared with N-gram analysis. Accepted types are .txt, .cpp, .h, .hpp, .c, .cc, .cxx, .py, .java, .js, .ts, .cs, .php, .html and .css (extension check is not case-sensitive).
- **Pre-processing:** case and whitespace normalisation of every file.
- **Similarity:** percentage calculation and category classification.
- **Fragment viewer:** matching fragments are shown side by side for each pair.
- **GUI:** folder selection, N-gram size selection, running the comparison, and searching, filtering, sorting and viewing results.
- **Persistent storage:** TXT, JSON and CSV reports.
- **Error handling:** unreadable files, empty input and failed exports.

1.2.2 Out of scope

- Comparison against internet sources, online repositories or a database of past submissions.
- Semantic or AST-level detection of the same algorithm written with a different structure.
- Binary, image or PDF documents.
- Multi-user, network or web-based deployment.
- Automatic disciplinary decisions: the tool only reports similarity, and the final judgement rests with a human reviewer.

1.2.3 Assumptions and constraints

- Input files are plain text and readable by the user running the program.
- The tool was developed and tested on Windows with a C++17 compiler (`std::filesystem` for file handling) and the Qt framework for the GUI.
- Similarity is a statistical indicator, not proof of plagiarism; common boilerplate such as `#include <iostream>` or `using namespace std;` can produce small amounts of legitimate overlap.

1.3 Background: N-Gram Similarity

1.3.1 N-grams and pre-processing

An N-gram is a contiguous sequence of N tokens taken from a text. In this project a token is a whitespace-separated word, so the N-grams are word-level. Before N-grams are generated, `TextProcessor` converts the text to lower case and collapses every run of spaces, tabs and line breaks into a single space. For $N = 3$, the statement `cout « x « endl;` has five tokens and therefore produces three overlapping N-grams: `cout « x`, `« x «` and `x « endl;`. A file with fewer than N words produces no N-grams.

1.3.2 Similarity measure

The similarity of two files A and B is computed from the sets of their distinct N-grams (the Jaccard index):

$$\text{Similarity}(\%) = \frac{|G(A) \cap G(B)|}{|G(A) \cup G(B)|} \times 100$$

where $G(A)$ and $G(B)$ are the sets of distinct N-grams of the two files. If neither file produces any N-gram, the similarity is defined as 0. Identical files give 100% and files with no N-gram in common give 0%. For example, if two files have 10 and 8 distinct N-grams, of which 4 are shared, the union contains $10 + 8 - 4 = 14$ N-grams and the similarity is $4/14 \times 100 \approx 28.6\%$.

1.3.3 Similarity categories

`PlagiarismReport::addResult` assigns each pair to a category using fixed thresholds (Table 1.1).

Similarity	Category	Meaning
below 30%	Low	Overlap is limited to common structure and boilerplate
30% to below 60%	Moderate	Noticeable shared code; review recommended
60% to below 80%	High	Large shared portions; needs attention
80% or more	Very High	Files are identical or nearly identical

Table 1.1: Similarity categories used by the report.

1.4 System Architecture and Design

1.4.1 Separation of concerns

The project follows a layered design. The back end contains all logic and has no dependency on the GUI, so it could also be driven from a command-line front end. The front end only collects input and displays output.

Layer	Modules	Responsibility
Presentation (front end)	Qt GUI, `SimilarityPieChart`	Folder selection, N-gram size selector, Check Similarity button, progress bar, summary cards, pie chart, search box, category filter, sort selector, results table, side-by-side fragment viewer, Export Results to CSV
Application logic (back end)	`ComparisonManager`	Coordinates the pipeline, runs every pair of files through it and returns the results
Processing	`FileManager`, `TextProcessor`, `NGramGenerator`, `SimilarityCalculator`	Finding and reading files, normalising text, generating N-grams, computing the similarity percentage and the matching fragments
Output and persistence	`PlagiarismReport`, `Result`	Storing results, assigning the similarity category, writing the TXT and JSON reports (the CSV export is triggered from the GUI)

Table 1.2: Layers of the system and the classes in each layer.

1.4.2 Modules

1. **File Manager** finds the input files in a folder (`findFiles`) and reads their contents (`readFile`).
2. **Text Processor** cleans and normalises the text (`processText`).
3. **N-Gram Generator** converts the processed text into overlapping word-level N-grams of the requested size, 3 by default (`generateNGrams`).
4. **Similarity Calculator** compares the N-grams of two files and returns a percentage (`calculateSimilarity`) and the list of shared fragments (`findMatchingNGrams`).
5. **Comparison Manager** runs every pair of files through the pipeline, reports progress through a callback and returns the results (`compareFiles`).
6. **Result Manager and Report Generator** are implemented together in the `PlagiarismReport` class, which stores `Result` objects, assigns each a category and writes the TXT and JSON reports.
7. **GUI** is the Qt desktop interface that ties everything together and adds the CSV export.

1.4.3 Processing flow

Figure 1.1 shows the processing flow of the back end, from the input files to the saved reports.

4.1 Overall program workflow

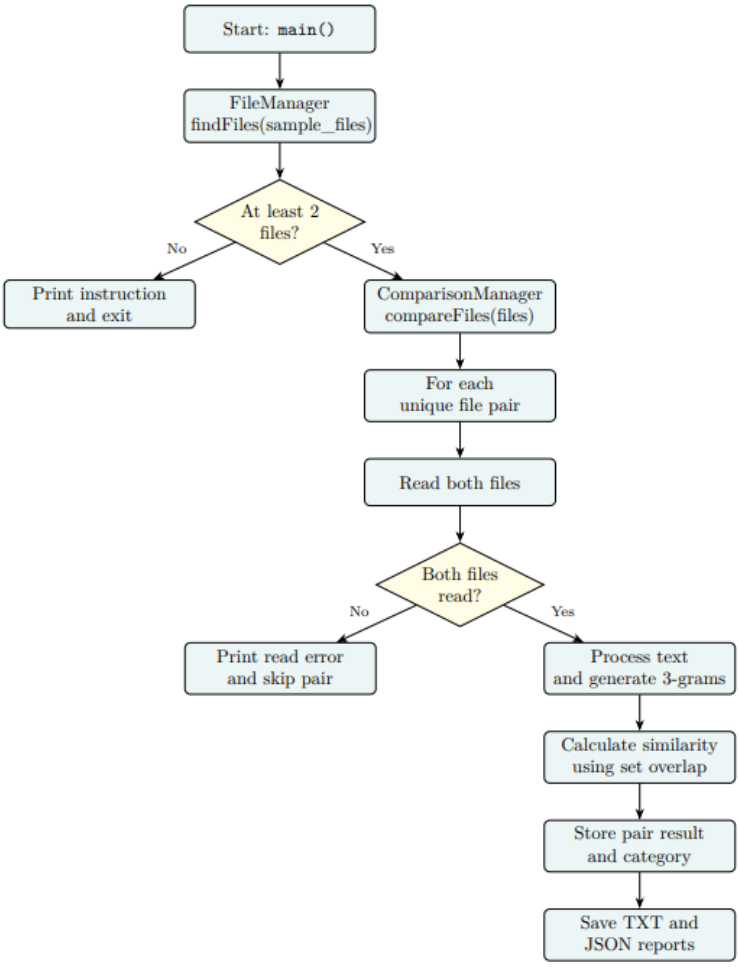


Figure 8: Actual console workflow implemented in `main.cpp` and `ComparisonManager.cpp`.

Figure 1.1: Processing flow of the back end, from the input files to the saved reports.

1. `FileManager::findFiles` collects the accepted files from the input folder.
2. If fewer than two files are found, the program prints an instruction and exits.
3. `ComparisonManager::compareFiles` takes each pair of files and reads them with `FileManager::readFile`.
4. `TextProcessor::processText` normalises the text of each file.
5. `NGramGenerator::generateNGrams` builds the N-grams (3 by default).
6. `SimilarityCalculator::calculateSimilarity` returns the similarity percentage for the pair.
7. `PlagiarismReport::addResult` stores the result with its category.
8. `saveReport` writes `plagiarism_report.txt` and `saveJsonReport` writes `plagiarism_report.js`

In the GUI, the Check Similarity button runs the same pipeline and shows the results on screen; the CSV file is written from the Export Results to CSV button.

1.5 Object-Oriented Architecture

1.5.1 Class design and associations

Each module is a separate class with one clear job. The only data the design stores is the list of results, which is private inside `PlagiarismReport` and can be changed only through `addResult`.

Class	Data	Public interface
FileManager	none	`findFiles(folderPath) : vector<string>; readFile(filePath) : string`
TextProcessor	none	`processText(text) : string`
NGramGenerator	none	`generateNGrams(text, n) : vector<string>`
SimilarityCalculator	none	`calculateSimilarity(gramsA, gramsB) : double; findMatchingNGrams(gramsA, gramsB) : vector<string>`
ComparisonManager	none	`compareFiles(files, n = 3, progressCallback) : vector<Result>`
PlagiarismReport	`-results : vector<Result>` (private)	`addResult(file1, file2, similarity, matchingNGrams); saveReport(fileName); saveJsonReport(fileName, n); getResults()`
Result (struct)	`file1, file2, similarity, category, matchingNGrams`	plain data carrier for one compared pair

Table 1.3: Classes, data and public interface of the back end.

1.5.2 Access modifiers

- **private:** the `results` vector in `PlagiarismReport`, so that results are added only through `addResult`, which also assigns the category.
- **public:** the small set of operations each class offers to the others (Table 1.3).
- `Result` keeps its fields public because it only carries data between `PlagiarismReport` and the report writers.

Listing 1.1: Encapsulation: the results can only be changed through addResult, which assigns the category.

```
class PlagiarismReport
{
private:
    vector<Result> results;

public:
    void addResult(string file1, string file2, double similarity,
                   vector<string> matchingNGrams);
    void saveReport(string fileName);
    void saveJsonReport(string fileName, int n = 3);
    vector<Result> getResults();
};
```

1.5.3 Composition, aggregation and inheritance

The back-end design is built on **composition** rather than inheritance. `ComparisonManager::compareF` creates local `FileManager`, `TextProcessor`, `NGramGenerator`, `SimilarityCalculator` and `PlagiarismReport` objects and delegates one task to each. `PlagiarismReport` owns a collection of `Result` objects; the diamond in Figure 1.2 marks this composition.

The project does **not** use inheritance: there is no class hierarchy between the project classes. Inheritance and runtime polymorphism are only planned (see Section 1.12), where the two report writers could sit behind an abstract `ReportWriter` base class with a virtual `write()` function.

The interface of `compareFiles` is designed for extension without modification. The N-gram size is a parameter with a default of 3, and the function accepts an optional progress callback of type `std::function<void(int, int)>`, so the back end reports progress without depending on Qt. A console front end can simply omit the callback.

Listing 1.2: Comparison Manager signature: defaults and an optional progress callback.

```
class ComparisonManager
{
public:
    vector<Result> compareFiles(
        vector<string> files,
        int n = 3,
        function<void(int, int)> progressCallback = nullptr
    );
};
```

1.5.4 UML class diagram

Figure 1.2 shows the class relationships of the back end.

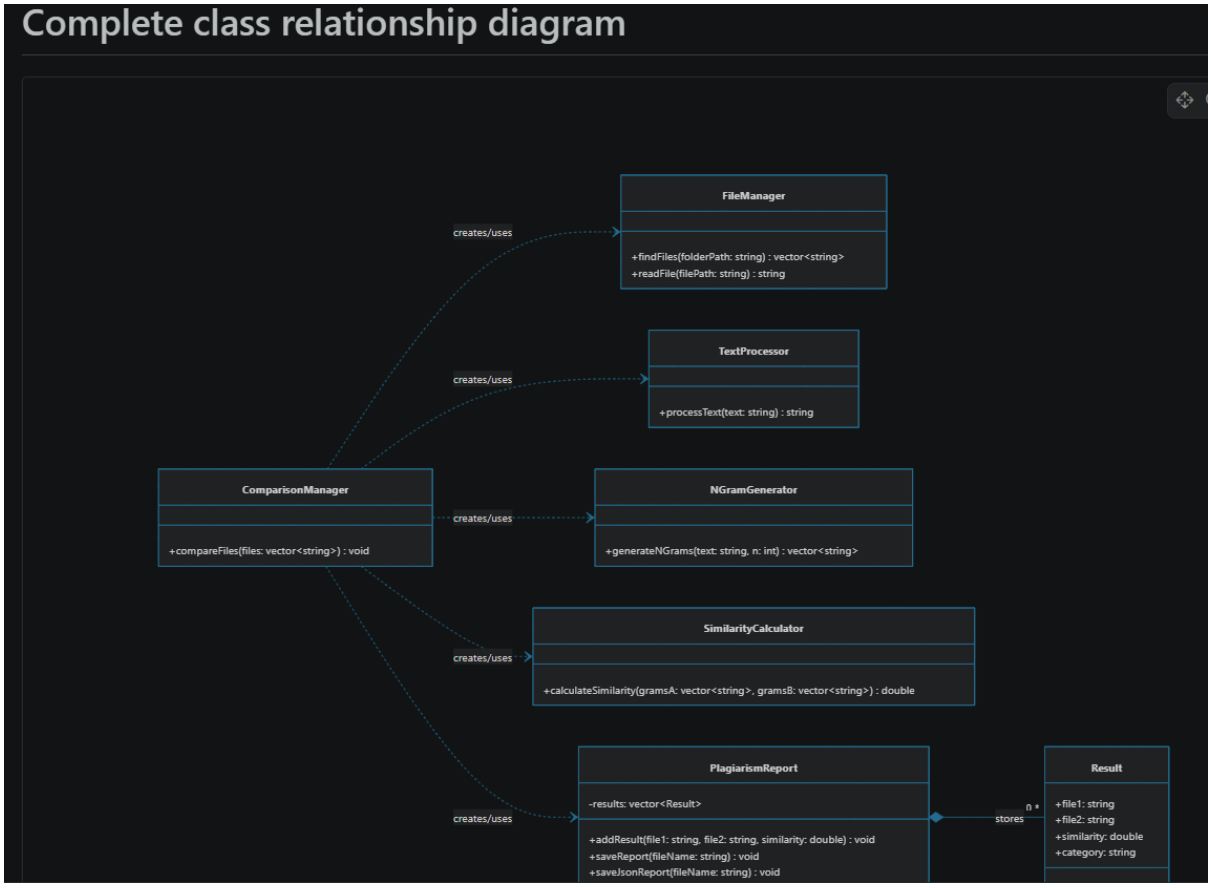


Figure 1.2: Class relationship diagram of the back end.

`ComparisonManager` has a "creates/uses" dependency on `FileManager`, `TextProcessor`, `NGramGenerator`, `SimilarityCalculator` and `PlagiarismReport`. `PlagiarismReport` is composed of zero or more `Result` objects (it "stores" them), so the results do not exist outside the report.

1.6 Advanced C++ Mechanics

1.6.1 Templates

The project relies on the C++ standard library's class templates throughout. A template is written once with a type parameter and then instantiated for each type it is used with.

- `vector<T>` is instantiated as `vector<string>` for the file list returned by `FileManager::findFile` and for the N-grams produced by `NGramGenerator::generateNGrams`, and as `vector<Result>` for the results held by `PlagiarismReport` and returned by `ComparisonManager::compareFiles`.
- `set<T>` is instantiated as `set<string>` inside `SimilarityCalculator::calculateSimilarity`, where each file's N-grams are converted to a set of distinct values so that the intersection and union sizes can be counted. It is also used in `FileManager::findFiles` for the list of accepted file extensions.
- `std::function<R(Args...)>` is instantiated as `function<void(int, int)>` for the progress callback of `compareFiles`. It can hold a lambda, a function pointer or

a functor. The GUI passes a lambda that updates the progress bar, and a console front end can pass nothing.

Because these templates are type-safe and resolved at compile time, they provide generic, reusable containers without run-time overhead or manual type casting.

1.6.2 Exception handling and error checking

Error cases are handled so that a bad input or a failed write does not crash the program or leave a damaged file.

- **Exception handling.** `FileManager::findFiles` scans the folder inside a `try` block. If the folder does not exist or cannot be read, `std::filesystem::directory_iterator` throws a `filesystem_error`. The `catch` block returns the files found so far (an empty list in that case) instead of letting the exception terminate the program.
- `FileManager::readFile` checks `is_open()` and returns an empty string for an unreadable file. `ComparisonManager::compareFiles` skips every pair that contains an empty text, so one unreadable file does not stop the comparison of the others.
- `NGramGenerator::generateNGrams` returns an empty list when N is not positive or when a file has fewer than N words.
- `PlagiarismReport::saveReport` and `saveJsonReport` check that the output file was opened before writing to it.
- With fewer than two input files, no empty comparison is run: the GUI shows a "Not Enough Files" message and the console driver prints an instruction and exits cleanly. If no folder has been chosen, the GUI asks the user to select one.
- In the fragment viewer, both files are reopened; if either cannot be opened, a "File Error" warning names the file.
- The CSV export writes through a `QSaveFile` and checks the stream status after writing. If writing failed, an "Export Failed" message box appears and the write is cancelled with `cancelWriting()`, so a half-written file is never saved. The file is committed with `commit()` only when the write succeeded, and a failed commit is reported in a second error dialog.

Listing 1.3: Exception handling in FileManager::findFiles.

```
try
{
    for (const auto& entry : filesystem::directory_iterator(
        folderPath))
    {
        // collect files with accepted extensions
    }
}
catch (const filesystem::filesystem_error&)
{
    return files;
}
```

1.6.3 Operator overloading

The project does not define any overloaded operators of its own. Operators are used in their predefined form and in the overloaded form provided by the standard library and Qt. `operator«` and `operator»` of the stream classes write and read data, for example `file « "- " « gram` in `PlagiarismReport::saveReport` and `ss » word` in `NGramGenerator::generateNGrams`. `set<string>` uses the `operator<` that the standard library provides for strings.

Results are ordered in the GUI with a comparator lambda passed to `std::sort` rather than an overloaded `operator<` on `Result`, because the results table can be sorted either way.

1.6.4 Memory management

The back end stores all data in standard containers and strings (`vector<string>` for file lists and N-grams, `vector<Result>` for the results). These release their memory automatically when they go out of scope, so there is no manual `new`/`delete` in the comparison pipeline. In the GUI, widgets are created with `new` and attached to a parent widget or layout, and Qt's parent–child ownership releases them when the parent window is destroyed.

1.7 Frontend and Backend Integration

1.7.1 Frontend

The GUI is built with the Qt framework and presents everything on a single dark-themed window (Figure 1.3).

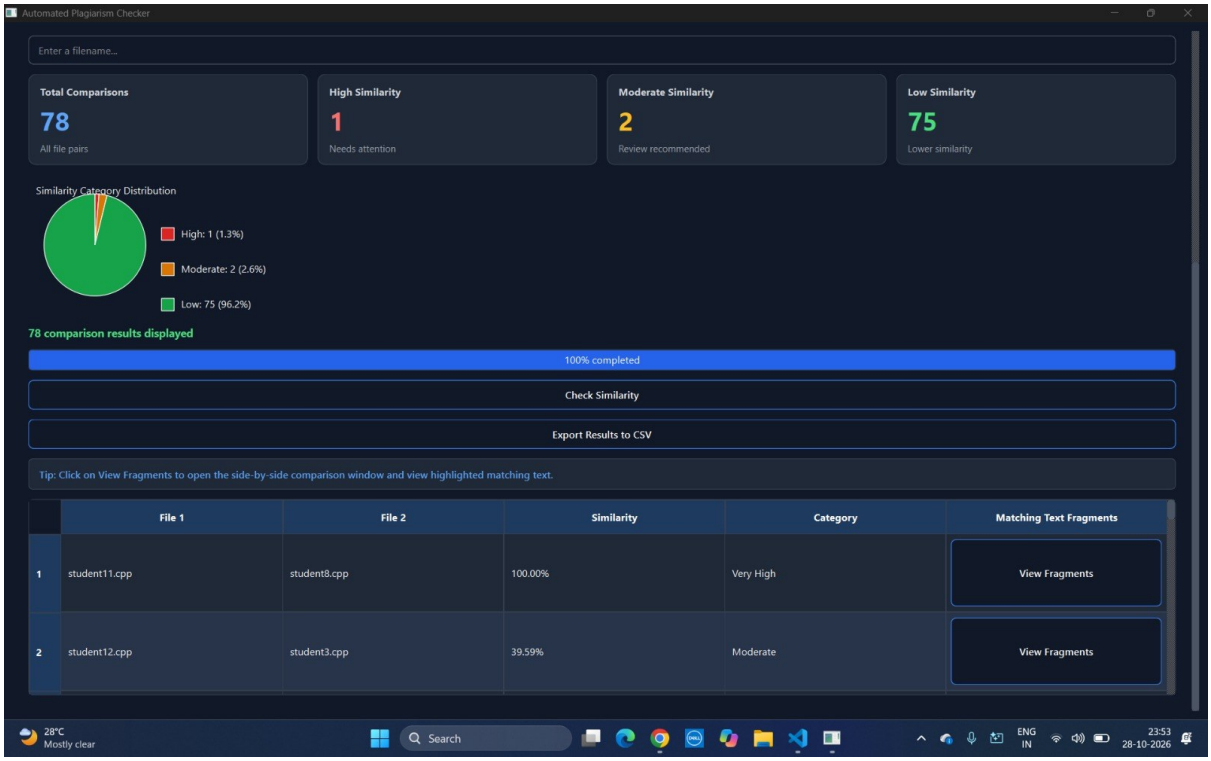


Figure 1.3: Main window after comparing 13 files (78 pairs).

- A **Select Folder** button chooses the folder to analyse, and a selector sets the N-gram size (2, 3 or 5 words, default 3). The **Check Similarity** button starts the comparison, and a progress bar and status message show how far it has got.
- Summary cards show the total number of comparisons and how many pairs fall in the High, Moderate and Low categories (High includes Very High).
- A pie chart shows the share of each category.
- A search box filters the table by file name, a category filter shows only High, Moderate or Low pairs, and a sort selector orders the pairs by similarity.
- A results table lists File 1, File 2, Similarity, Category and a **View Fragments** button for every pair.
- **Export Results to CSV** saves the results as a spreadsheet-friendly file.

View Fragments opens a side-by-side window in which the matching text is highlighted in both files (Figure 1.4).

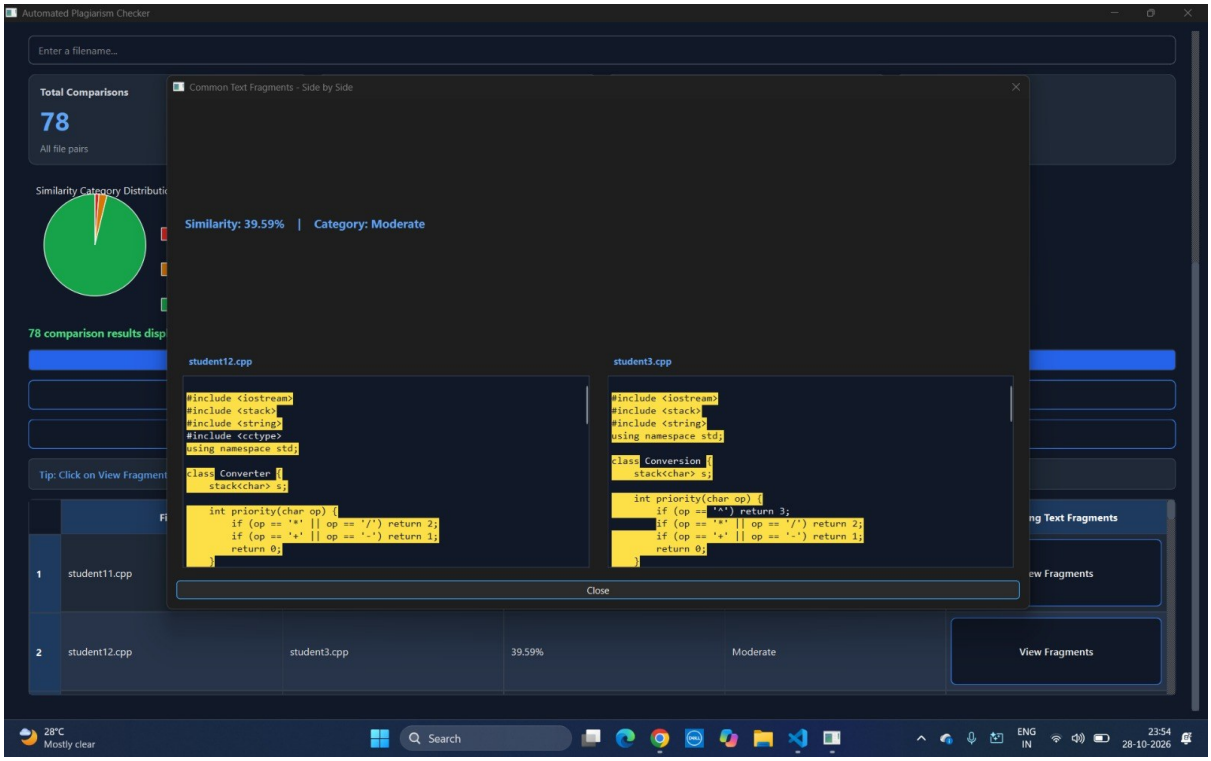


Figure 1.4: Side-by-side fragment viewer for student12.cpp and student3.cpp (39.59%, Moderate).

1.7.2 Backend

The back end is standard C++ code that finds and reads the files, processes the text, generates the N-grams and calculates the similarity. The GUI calls `compareFiles` on the Comparison Manager, passing the files found, the selected N and a progress callback, and receives the list of results; it never performs the comparison itself. `compareFiles` also writes the TXT and JSON reports before returning. This separation means the logic can be tested independently of the interface.

1.7.3 Persistent storage and file I/O

Results are saved to disk in three formats (Table 1.4). The TXT and JSON reports are written automatically at the end of every comparison, in the program's working directory, and are overwritten by the next run. The CSV file is written only when the user clicks Export Results to CSV and chooses a location.

Format	File	Written by	Purpose
TXT	`plagiarism_report.txt`	`PlagiarismReport::saveReport`	Human-readable report with file pair, similarity, category and matching fragments
JSON	`plagiarism_report.json`	`PlagiarismReport::saveJsonReport`	Machine-readable output with project name, N-gram size and all results; quotes, backslashes, newlines, carriage returns and tabs are escaped
CSV	`plagiarism_results.csv`	GUI export button (`QSaveFile`)	Spreadsheet-friendly table: File 1, File 2, Similarity (%), Category, Matching Text Fragments (joined with semicolons)

Table 1.4: Persistent report formats.

Figure 1.5 shows how a result is stored, categorised and written to the TXT and JSON reports.

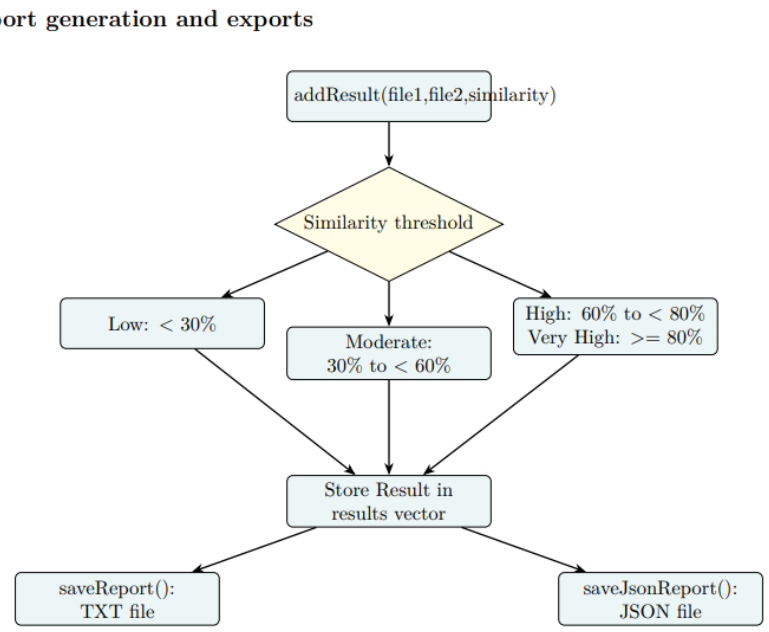


Figure 1.5: How a result is stored, categorised and written to the TXT and JSON reports.

The Export Results to CSV button opens a standard save dialog with `plagiarism_results` as the suggested file name (Figure 1.6).

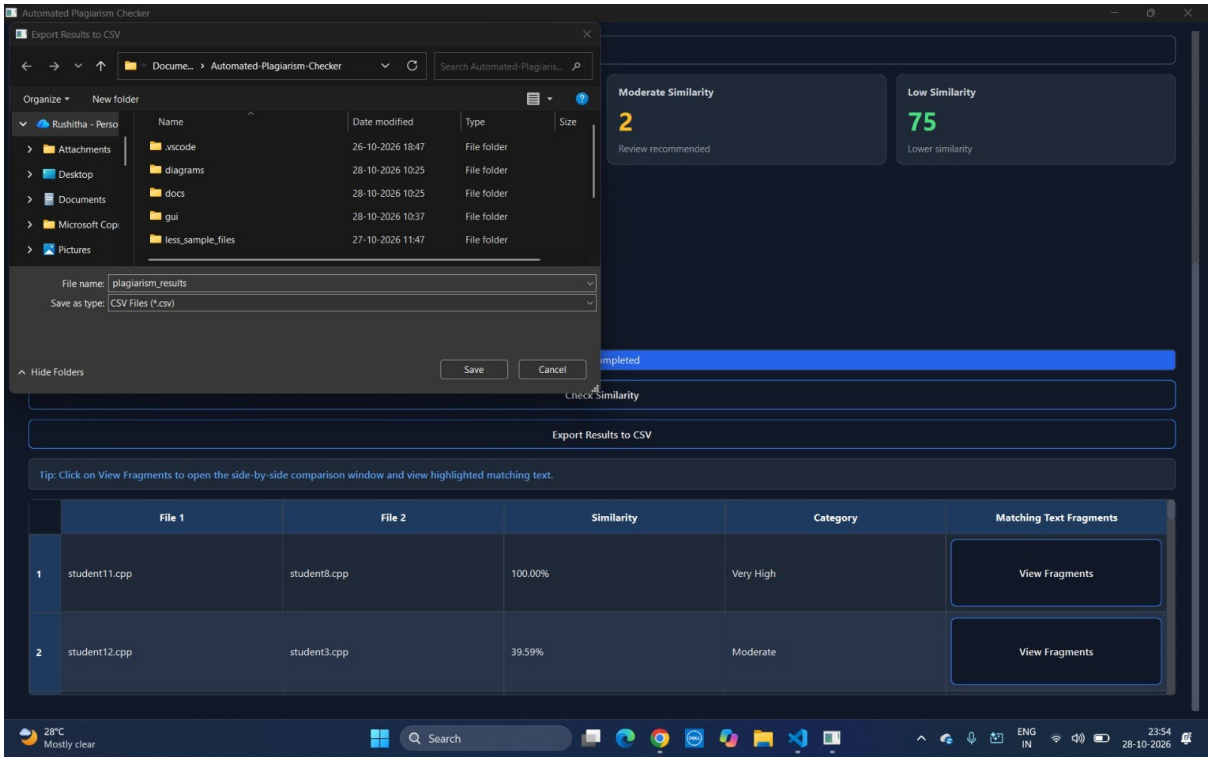


Figure 1.6: Export Results to CSV save dialog.

The three report files as they appear after a run are shown in Figures 1.7, 1.8 and 1.9.

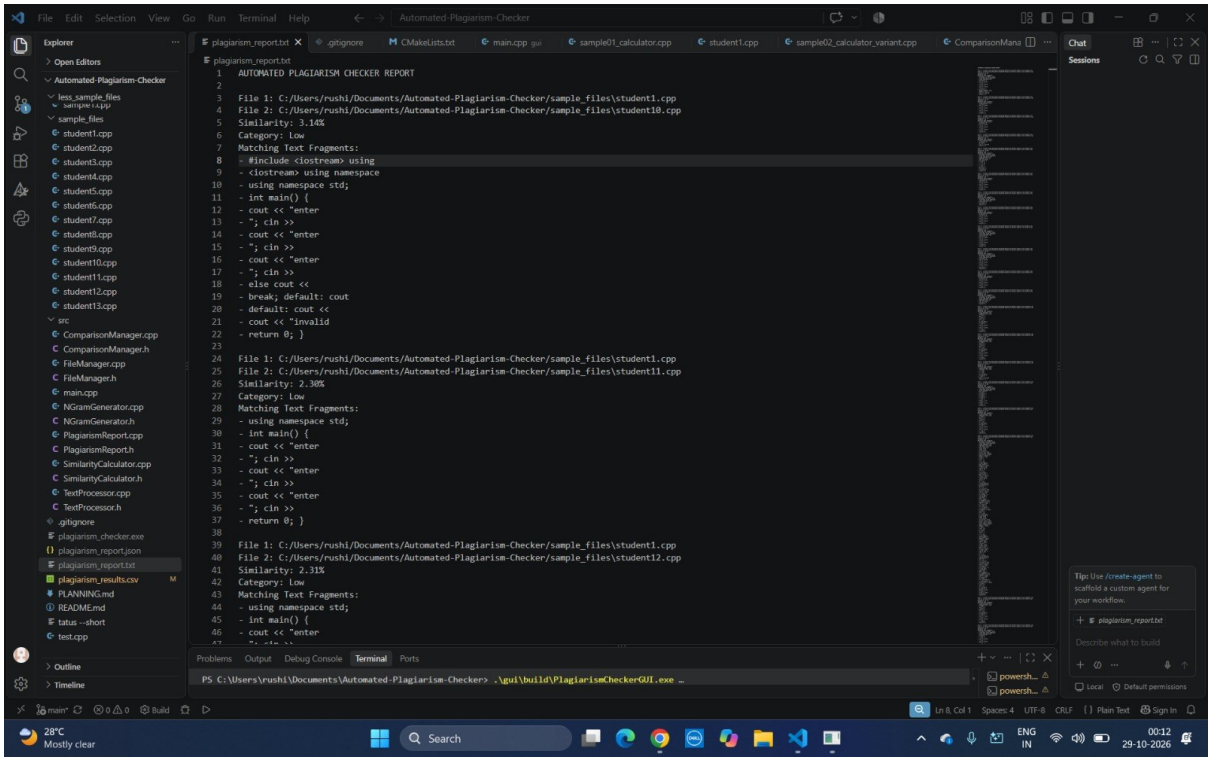


Figure 1.7: plagiarism_report.txt, with one block per file pair (files, similarity, category and matching fragments).

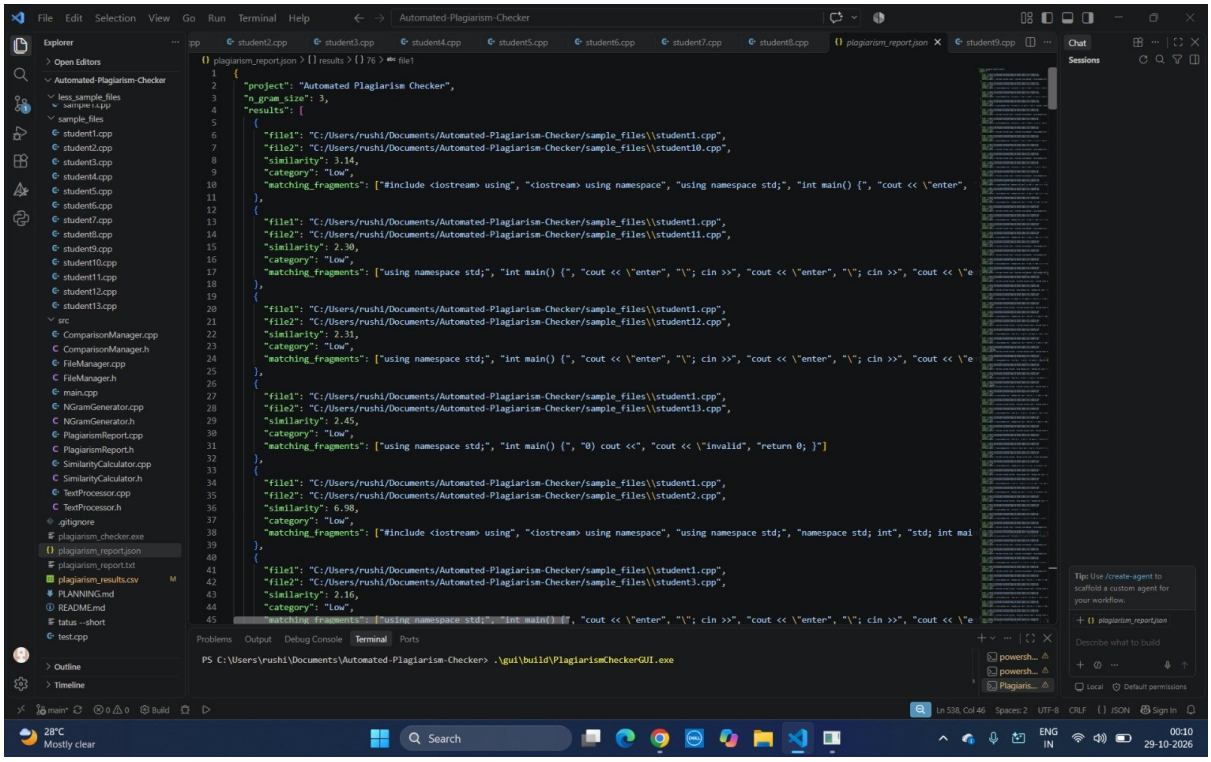


Figure 1.8: plagiarism_report.json, showing the project name, n_gram_size of 3 and the results array.

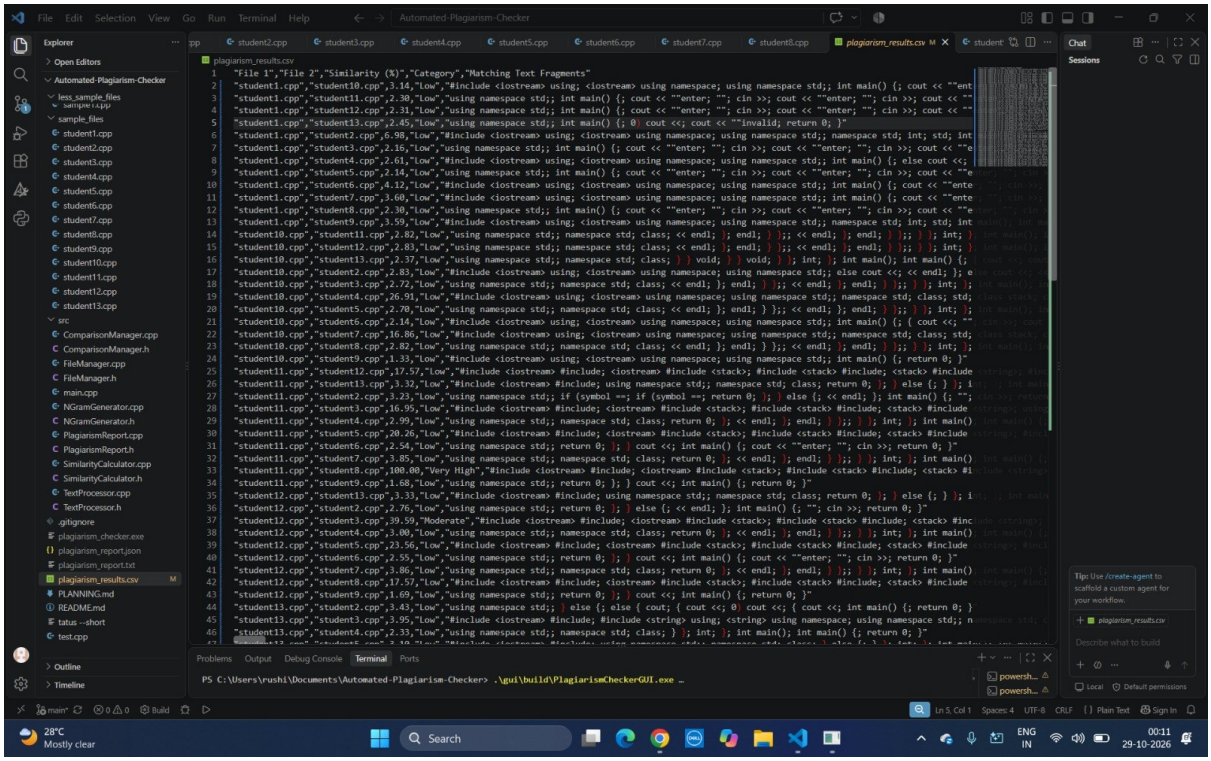


Figure 1.9: plagiarism_results.csv, one row per pair (student11 and student8 at 100.00, Very High; student12 and student3 at 39.59, Moderate).

The generated report files are listed in `.gitignore` together with the build output, so that only source code, and not generated artefacts, is tracked in the repository.

1.8 Implementation Summary

Item	Detail
Language	C++ (C++17, `std::filesystem`)
GUI framework	Qt Widgets (`QFileDialog`, `QMessageBox`, `QTableWidget`, `QTextStream`, `QSaveFile`)
Algorithm	Word-level N-gram analysis; N selectable (2, 3 or 5), default 3; similarity = shared ÷ combined distinct N-grams (Jaccard index)
Input	Text and source code files (.txt, .cpp, .h, .hpp, .c, .cc, .cxx, .py, .java, .js, .ts, .cs, .php, .html, .css) from a user-selected folder
Output	On-screen results (searchable, filterable, sortable); TXT, JSON and CSV reports
Repository layout	`src/` (back-end classes), `gui/` (Qt front end), `sample_files/` and `less_sample_files/` (test data), `docs/`, `diagrams/`, planning and README documents

Table 1.5: Implementation summary.

1.9 Version Control (Git)

The project is maintained in a public Git repository (Automated-Plagiarism-Checker) hosted on GitHub.

- **Planning first:** a `PLANNING.md` file records the project overview and the planned modules, and `README.md` summarises the project, so the design intent is documented in the repository itself.
- **Incremental commits:** work was committed in small, focused steps, one change or feature at a time, with descriptive commit messages.
- **Repository hygiene:** a `.gitignore` excludes the GUI build directory, the compiled executable and the generated report files.
- **Reviewing changes:** `git diff` and `git status -short` were used before committing.
- **Collaboration:** each member works on separate modules or features and merges them into the shared repository.

Figure 1.10 shows the public repository on the main branch, with 51 commits. The latest commit messages describe one change each, for example “Display matching text fragments side by side with highlighting”, “Add comparison progress indicator” and “Clear stale results before new comparisons”.

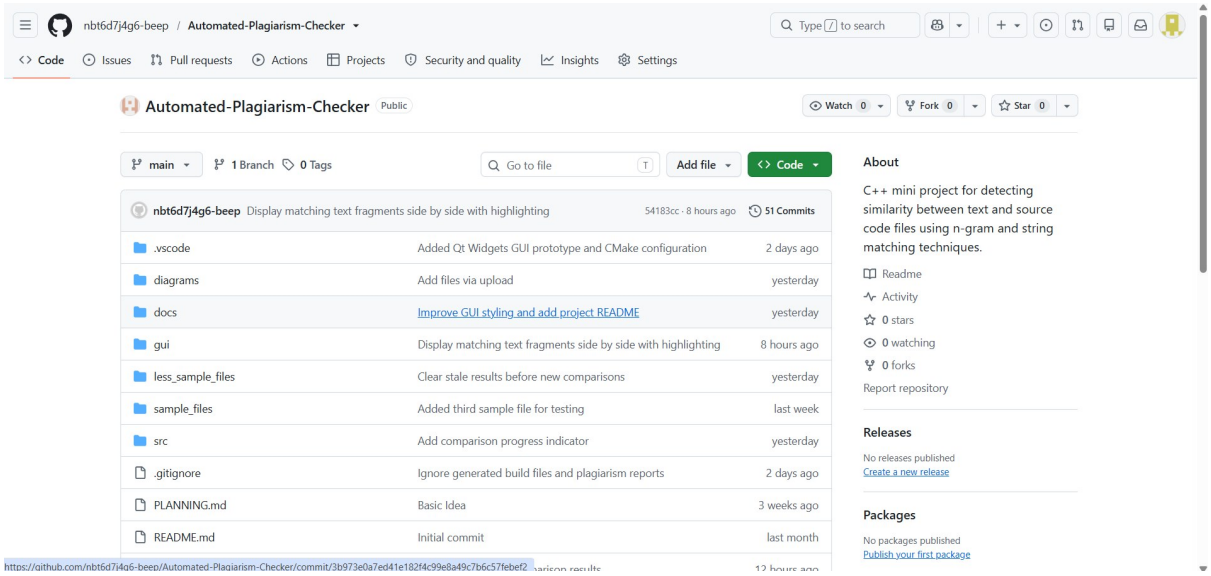


Figure 1.10: The public GitHub repository on branch main, with 51 commits.

1.10 Testing and Results

1.10.1 Test data

The application was tested on a `sample_files` folder of 13 small C++ programs (student1.cpp to student13.cpp), using the default N-gram size of 3. Several of them solve

the same task, such as converting an expression with a stack, so they naturally share structure. With 13 files the tool compares 78 unique pairs.

1.10.2 Observed results

Measure	Value
Files compared	13
File pairs analysed	78
N-gram size	3
Pairs rated Very High (80% or more)	1 (1.3%)
Pairs rated High (60% to below 80%)	0 (0%)
Pairs rated Moderate (30% to below 60%)	2 (2.6%)
Pairs rated Low (below 30%)	75 (96.2%)

Table 1.6: Observed results on the 13-file test set.

Percentages are rounded to one decimal place. Key findings (Figure 1.3):

- `student11.cpp` and `student8.cpp`: 100% (Very High). The files are identical in content, and the tool placed this pair at the top of the results table.
- `student12.cpp` and `student3.cpp`: 39.59% (Moderate). The side-by-side viewer (Figure 1.4) highlights the same `#include` lines, the same `using namespace std;` line, a class that wraps a `stack<char>` and the same `priority(char op)` function. The two classes use different names (`Converter` and `Conversion`), which is a typical sign of partly reused code.
- `student4.cpp` and `student7.cpp`: 36.54% (Moderate). Besides the common boilerplate, their matching fragments show the same stack class structure: `class stack`, `top = -1;` and the member functions `empty()`, `full()`, `push(int x)` and `pop()`.
- The remaining 75 pairs are Low. Their overlap is limited to common program structure and standard C++ boilerplate, which is expected between independent programs.

1.10.3 Evaluation

The results show that the tool (a) detects exact duplicates reliably, (b) picks up partially copied code even when class names have been changed, (c) keeps independent programs in the Low category, and (d) explains each result through highlighted matching fragments so that a reviewer can verify a finding quickly.

1.10.4 Error and edge-case testing

Scenario	Expected behaviour
No folder selected	Warning asking the user to select a folder first; no comparison is run
Folder with fewer than two files	"Not Enough Files" message; no comparison is run
Unreadable or missing file	File is skipped (its pairs produce no result); the program continues with the other files
Export to a location that cannot be written	"Export Failed" dialog; incomplete file is not saved
Identical files	100% similarity, Very High category (student11.cpp and student8.cpp)
Files differing only in indentation, blank lines or extra spaces between words	Still detected as highly similar because of text normalisation
File with fewer than N words	No N-grams are produced for that file, so its similarity with other files is 0

Table 1.7: Error and edge-case testing.

1.11 Limitations

- The method is lexical: code with the same logic written using a different structure, or with heavily renamed identifiers, may score lower than it deserves.
- Tokens are separated only by whitespace, so adding or removing spaces around operators and brackets (for example `a+b` versus `a + b`) changes the N-grams.
- Common boilerplate raises the baseline similarity between all C++ programs; the Low category absorbs this.
- Only files supplied by the user are compared; there is no comparison with external sources.
- Similarity thresholds (30%, 60% and 80%) are fixed in `PlagiarismReport::addResult` and may need adjustment for different assignment types or file lengths.
- `findMatchingNGrams` does not remove repeated fragments, so the same fragment can appear more than once in a pair's report.
- Every file is read and processed again for each pair in which it takes part, and the comparison runs on the GUI thread, so very large folders will be slow.
- `saveReport` and `saveJsonReport` return silently if their output file cannot be opened; the user is not notified.

1.12 Future Enhancements

- Token-based comparison that ignores identifier names, so that renaming variables no longer hides copying.
- Adjustable category thresholds in the GUI.

- Boilerplate exclusion (an ignore list for starter code supplied by the instructor).
- Removal of repeated matching fragments, and processing each file only once (caching its N-grams) with the comparison running in a worker thread.
- Additional export formats such as HTML and PDF.
- An abstract report-writer interface with virtual functions, so new formats need only a new derived class.
- Language-aware (token-based or syntax-tree) analysis and comparison against an archive of past submissions.

1.13 Conclusion

The Automated Plagiarism Checker meets its goal of providing a fast, transparent and offline way to compare a set of text or source code files. It combines an N-gram similarity engine with a clean object-oriented structure: independent classes with encapsulated data, a composition-based design, careful error handling around file input and output, and container-based memory management. A Qt interface sits on top of a separate C++ back end, and results are persisted as TXT, JSON and CSV reports. Testing on 13 sample files (78 pairs, N = 3) flagged one identical pair at 100%, placed two pairs (39.59% and 36.54%) in the Moderate band and kept the remaining 75 pairs Low. The project also demonstrates sound engineering practice through documented planning and incremental Git commits.

1.14 References

1. B. Stroustrup, *The C++ Programming Language*, 4th ed., Addison-Wesley, 2013.
2. cppreference.com, C++ Standard Library and `std::filesystem` documentation.
3. Qt Documentation, `QFileDialog`, `QMessageBox`, `QTextStream` and `QSaveFile` classes, https://doc.qt.io.
4. Git Documentation, https://git-scm.com/doc.
5. P. Jaccard, "The distribution of the flora in the alpine zone," *New Phytologist*, vol. 11, no. 2, pp. 37–50, 1912.
6. A. Z. Broder, "On the resemblance and containment of documents," in *Proc. Compression and Complexity of Sequences*, IEEE, 1997, pp. 21–29.
7. Course lecture notes and lab manual, Object-Oriented Programming with C++.

Appendix A: Sample Output (TXT Report Extract)

The first entry of `plagiarism_report.txt` generated for the 13-file test set (the folder path is shortened to "..."):

```
AUTOMATED PLAGIARISM CHECKER REPORT

File 1: .../sample_files\student1.cpp
File 2: .../sample_files\student10.cpp
Similarity: 3.14%
Category: Low
Matching Text Fragments:
- #include <iostream> using
- <iostream> using namespace
- using namespace std;
- int main() {
- cout << "enter
- "; cin >>
- cout << "enter
- "; cin >>
- cout << "enter
- "; cin >>
- else cout <<
- break; default: cout
- default: cout <<
- cout << "invalid
- return 0; }
```

Appendix B: Repository Files

- `PLANNING.md` – project overview and module plan
- `README.md` – short project description
- `.gitignore` – excludes build output and generated reports
- `src/` – `ComparisonManager`, `FileManager`, `NGramGenerator`, `PlagiarismReport`, `SimilarityCalculator`, `TextProcessor` and `main.cpp`
- `gui/` – Qt front end (`main.cpp`, `SimilarityPieChart`)
- `sample_files/`, `less_sample_files/` – test inputs
- `docs/`, `diagrams/` – documentation and diagram sources
- `plagiarism_report.txt`, `plagiarism_report.json`, `plagiarism_results.csv` – generated reports